

Report on zero bubble and 1F1B

<https://gemini.google.com/share/73d9b731da70>

Even with the 1F1B schedule, the cluster is not perfectly utilized 100% of the time.

1. The "Fill" and "Drain" Phases

Your intuition describes the timeline perfectly:

- **The Warm-up (Fill):**
 - At $T=0$, only **GPU 1** is working (on the first microbatch).
 - GPU 2, 3, and 4 are sitting idle, waiting for data to trickle down.
 - This is "wasted time" at the start.
- **The Steady State (Middle):**
 - This is the "zipper" phase we discussed. Everyone is busy. GPU 1 is processing Microbatch 10 while GPU 4 is processing Microbatch 2. This is the efficient part.
- **The Cool-down (Drain):**
 - At the very end, **GPU 1** finishes its last microbatch and goes idle.
 - However, **GPU 4** is still working on the backward pass for that last microbatch.
 - This is "wasted time" at the end.

2. The Shape of the Waste: The Parallelogram

If you visualize the active time of all GPUs on a chart, it doesn't look like a perfect rectangle. It looks like a **parallelogram**.

- The **empty triangles** at the top-left (start) and bottom-right (end) represent the **Pipeline Bubble**.
- This idle time effectively lowers the total computing power of your cluster.

3. The Math: Why we need many Microbatches (\$M\$)

There is a famous formula for the efficiency of a pipeline:

$$\text{Efficiency} \approx \frac{M}{M + P - 1}$$

- M = Number of Microbatches (e.g., 64 chunks)
- P = Number of Pipeline Stages (e.g., 8 GPUs)

The Logic:

If you only have a few microbatches (small M), the "Fill" and "Drain" phases dominate the time. The GPUs spend more time waiting for the pipeline to fill up than actually working.

If you have many microbatches (large M), the "Steady State" lasts a long time, and the "Fill/Drain" waste becomes negligible (amortized).

4. How the "Zero Bubble" Paper Fixes This

This brings us back to why the **Zero Bubble** paper is so clever.

In standard 1F1B, those start and end gaps are empty. The GPU is literally doing nothing.

The Zero Bubble method says: **"Hey, during the 'Drain' phase, GPU 1 is sitting idle because it finished all its Forward passes. Why don't we let it calculate the Weight Gradients (∇W) for the old batches *now*?"**

By pushing the "lazy" work (∇W) into those specific idle slots at the beginning and end, it effectively turns the parallelogram back into a rectangle, reclaiming that wasted time.

Summary

- **GPipe:** Huge Bubbles (Fill ALL $\rightarrow$ Drain ALL).
- **1F1B:** Smaller Bubbles (Fill Pipeline Depth $\rightarrow$ Steady State $\rightarrow$ Drain Pipeline Depth).
- **Zero Bubble:** "Fills" the bubbles with deferred ∇W tasks.

Would you like to know how we choose the **optimal number of microbatches** to balance this bubble waste against memory usage?

Choosing the optimal number of microbatches (M) is a balancing act between **Speed** (minimizing bubbles) and **Stability** (Math/Memory constraints).

<https://gemini.google.com/share/9c133d2bd659>

Why 1F1B is safe:

1. The "Frozen Weights" Rule

You might be imagining that as soon as a Backward pass finishes for Microbatch 1, the GPU updates its weights immediately.

If that were true, you would be right—it would break accuracy. If GPU 1 updated its weights after Microbatch 1, then Microbatch 2 (which is already halfway through the pipeline) would be using "old" weights on some layers and "new" weights on others. That would be a disaster.

But that is not what happens.

In 1F1B, the weights (W) stay **exactly the same** for the entire duration of the Global Batch (the sum of all microbatches).

- **Microbatch 1 Forward:** Uses Weights W_t
- **Microbatch 2 Forward:** Uses Weights W_t (same weights!)
- **Microbatch 1 Backward:** Calculates gradients based on W_t
- **Microbatch 2 Backward:** Calculates gradients based on W_t

2. Gradient Accumulation

Instead of applying the update immediately, the GPU has a temporary "bucket" called the **Gradient Accumulator**.

When the backward pass for Microbatch 1 finishes, the GPU doesn't touch the main weights. It just throws the calculated gradients into the bucket.

$$\text{Bucket} = \text{Bucket} + \nabla W_{\{\text{Microbatch 1}\}}$$

When Microbatch 2 finishes:

$$\text{Bucket} = \text{Bucket} + \nabla W_{\{\text{Microbatch 2}\}}$$

Only after **ALL** microbatches (e.g., all 100 chunks) are finished does the optimizer wake up, look at the total sum in the bucket, and update the weights **once**.

3. Synchronous vs. Asynchronous (The "Stale" Trap)

You are right to be suspicious, because there **is** a version of pipeline parallelism that compromises accuracy, but 1F1B is not it.

- **PipeDream (The "Dirty" Way):** An older method (around 2018/2019) tried to update weights immediately to go faster. This created "Stale Weights" (calculating gradients using a version of the model that no longer exists). This **did** lower accuracy.
- **1F1B / GPipe (The Standard Way):** This is **Synchronous**.¹ It forces the system to wait until the end of the global batch to update. It guarantees the math is perfect, at the cost of having to manage that schedule carefully.

The Zero Bubble Trick:

- **Old way:** GPU 2 calculates ∇W *and* ∇X together, then sends ∇X . (GPU 1 waits a long time).
- **New way:** GPU 2 calculates ∇X **first**, sends it immediately over the wire to GPU 1, and *then* calculates ∇W while GPU 1 is already busy working. Layer $n-1$ is **completely paralyzed** until it receives ∇X from Layer n .

the superiority of 1F1B over GPipe

The 1F1B schedule is better than GPipe primarily because of **Memory Efficiency**, not just raw speed.

If you run both with the exact same number of microbatches (M), they actually have the **exact same** speed and bubble size.

The problem is: **GPipe often crashes your GPU memory before you can reach that speed.**

Why do we don't need prefetch and cache scenario?

The standard architecture uses a **Producer-Consumer** model.

- **Producer (CPU):** Reads files from disk, turns text into numbers (tokenization),

and puts them in a Queue (Buffer).

- **Consumer (GPU):** Takes from the Queue and calculates gradients.

The Math of Safety:

For Large Language Models (LLMs), the math is overwhelmingly in the CPU's favor:

- **GPU Task:** Multiply giant matrices (\$100 \times 100\$ billion parameters). This takes a "long" time (e.g., 500ms per batch).
- **CPU Task:** Read a few kilobytes of text integers from RAM. This is instant (e.g., 1ms).

Because the CPU is **500x faster** at its job than the GPU is at its job, the "Service Counter" (Prefetch Buffer) is always overflowing with prepared batches. The GPU never has to ask; it just grabs.

continuous communication

"continuous communication" is technically known as **Computation-Communication Overlap**.

You are absolutely right to ask this, because it is the primary way we fix the "waiting" problem we discussed earlier.

In the standard "Stop-and-Wait" approach, the timeline looks like a traffic light: Compute (Red) $\rightarrow$ Communicate (Green) $\rightarrow$ Compute (Red).

In the "Continuous" approach, the goal is to make communication flow like a background river while the computation happens on the shore.

Here is the breakdown of how this magic trick is performed.

1. The Trick: Two Separate "Lanes" (CUDA Streams)

To understand this, you have to look at the GPU hardware. A GPU is not just one big brain; it has separate engines for separate tasks.

- **Compute Engine (SMs):** Does the math (Matrix Multiplication).
- **Copy Engine (DMA):** Moves data over the wire (NVLink/PCIe).

The Continuous Concept:

Instead of the Compute Engine pausing to tell the Copy Engine to work, we give them independent instructions (using CUDA Streams) so they run simultaneously.

- **Lane 1 (Math):** "Calculate Gradient for Layer 10... Done. Now Layer 9... Done."
- **Lane 2 (Comm):** "Send Gradient 10... (Still sending)... Send Gradient 9..."

2. The Mechanism: "Bucketing" (The Conveyor Belt)

You cannot send every single parameter instantly (too much overhead). You also don't want to wait for *all* parameters to be ready (too much latency).

The solution is **Gradient Bucketing** (used in PyTorch DDP and DeepSpeed).

Imagine the model is a stack of 100 boxes (layers).

1. **Backward Pass Starts:** The GPU calculates gradients for the top layer (Layer 100).

2. **The Bucket Fills:** It throws those numbers into a small "Bucket" (buffer).

3. **The Trigger:** As soon as the bucket is full (e.g., 25MB), the system **immediately** hands it to the Copy Engine to ship it out.

4. **Continuous Flow:**

- While the Copy Engine is shipping the "Layer 100 Bucket," the Compute Engine is *already* calculating Layer 99 and 98.

- By the time Layer 95 is calculated, the "Layer 100 Bucket" has finished arriving at the destination.

3. Why it feels "Continuous"

If you look at the network traffic monitor during training with this feature enabled:

- **Without Overlap:** You see spikes. *Silence... Silence... SPIKE (Sync)... Silence.*
- **With Overlap:** You see a **flat, continuous line** of traffic. The data is trickling out constantly in the background throughout the entire backward pass.

4. The "Hiding" Factor

The ultimate goal is to **hide** the communication time completely.

- **Math Time:** 500ms

- **Comm Time:** 100ms

If you do them sequentially: Total = 600ms.

If you overlap them: Total = 500ms. (The 100ms of communication happened during the 500ms of math).

The Limit:

If your communication is slower than your math (e.g., Math 500ms, Comm 800ms), you cannot hide it all. You will hide 500ms of it, but the GPU will still have

to wait for the final 300ms at the end. This is when we say the system is "Communication Bound."

Summary

"Continuous Communication" effectively means **Asynchronous pipelining of the network traffic**.

- **Old Way:** Calculate *everything*, then sync *everything*.
- **New Way:** Calculate a *chunk*, ship that *chunk* immediately, and keep calculating the next *chunk*.

This ensures the network cables are busy 100% of the time, rather than just at the end of the batch.

Bucketing is the primary optimization for the "Gradient Synchronization" step in Data Parallelism.

from AutoDDL:

There are other techniques for reducing communication overhead that are orthogonal to AutoDDL. For example, gradient sparsification [47], [48], [49] only communicates the largest k gradients to reduce the communication volume, while gradient quantization [50], [51], [52] reduces the number of bits for the gradient values by using lower precision. These communication-reducing techniques can be integrated into AutoDDL by factoring in the reduced communication volume when evaluating the performance of different parallelization strategies

During the training of most LLMs, each token is typically encountered only once. However, employing data **caching** remains crucial to mitigate I/O bottlenecks during data loading. This strategy involves prefetching training data from slower backend storage to faster **cache** storage. Alluxio [89] and JuiceFS [90] enhance LLM training by efficiently **caching** training data from underlying storage systems such as HDFS or object storage. Quiver [91] supports the transparent reuse of **cached** data across multiple jobs and users operating on the same dataset. Fluid [92] leverages Alluxio for data **caching**, incorporating a mechanism that enables on-the-fly autoscaling of the **cache** based on I/O conditions.

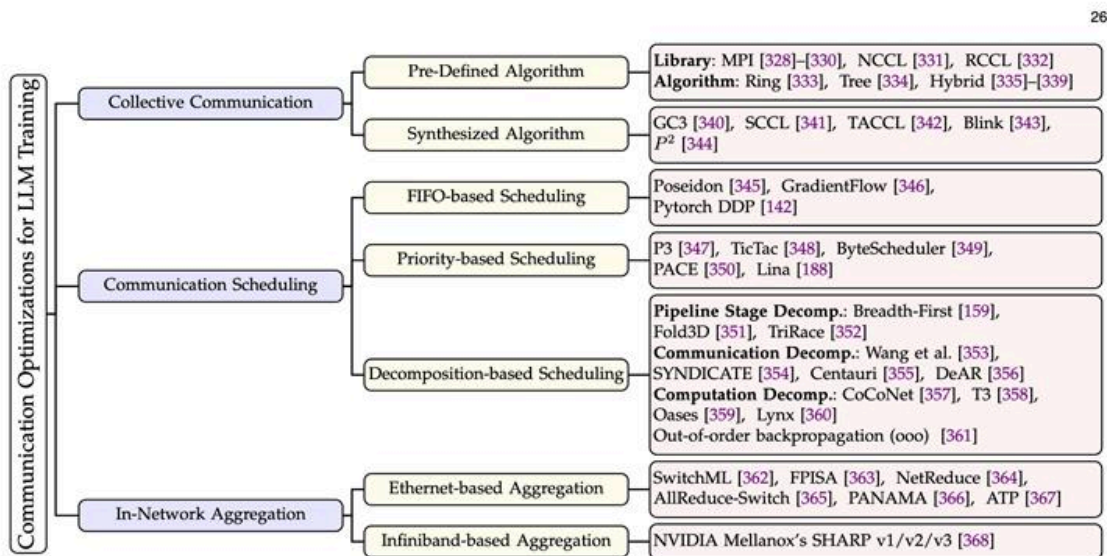


Fig. 14: Studies on communication optimizations for distributed LLM training.

Mobius optimizes communication through prefetching and cross-mapping to reduce overhead and contention. Yuan et al. [301]

This is good

<https://gemini.google.com/share/e1932e7f07ec>

Here is the consolidated table of all the timing estimates we discussed.

The Baseline Assumption: We are using a large model (like Llama 70B) where a single **Forward Pass (\$F\$)** takes **100ms**.

1. Computation Times (The GPU Math)

This is the "Work" that dictates the speed of the pipeline.

Operation	Time Estimate	Relative to Forward	Description
Forward Pass (\$F\$)	100 ms	1.0x	Matrix Multiplications. The baseline unit.
Backward Input (\$\nabla X\$)	~110 ms	1.1x	Calculates error for previous stage. Urgent .
Backward Weight (\$\nabla W\$)	~110 ms	1.1x	Calculates weight updates. Lazy (Zero Bubble fills idle time with this).
Total Backward	~220 ms	2.2x	The sum of ∇X and ∇W .
Optimizer Step	~30 ms	0.3x	Adam/SGD update. Usually memory-bound.
Total Iteration	~350 ms	3.5x	Forward + Backward + Optimizer.

2. Communication Times (Moving Data between GPUs)

Assuming a standard Microbatch size of ~134 MB. This is the time to send ∇X to the previous stage.

Connection Type	Time Estimate	Verdict	Impact on Pipeline
NVLink (Intra-Node)	~0.3 ms	Instant	Perfectly hidden. No bubble.
InfiniBand (Inter-Node)	~3.0 ms	Fast	Easily hidden. Negligible bubble.
Cloud Ethernet (10Gbps)	~134.0 ms	Slow	Catastrophic . Slower than compute. Pipeline stops.

Network Latency	~0.1 ms	Fixed Cost	The "Handshake" cost paid per packet (Fixed via Bucketing).
------------------------	----------------	-------------------	---

3. Data Loading Times (Getting Data Ready)

Assuming a 100 MB Batch. The race is to finish this before the 100ms Forward pass ends.

Storage Tier	Time Estimate	Verdict	Can we hide it?
GPU Cache (HBM)	0.05 ms	Instant	Yes.
CPU RAM \$to\$ GPU	~2.0 ms	Instant	Yes. (This is the standard Prefetch).
NVMe SSD \$to\$ RAM	~20.0 ms	Safe	Yes. (20ms << 100ms).
SATA SSD \$to\$ RAM	~200.0 ms	Risk	No. (200ms > 100ms). Causes stalling.
HDD (Spinning)	~1,000 ms	Fail	No. GPU sleeps for 90% of the time.

Summary of the Race

To maintain a perfect **Zero Bubble** or **1F1B** schedule (100% GPU Utilization), the following inequality must hold true:

$$T_{\text{Communication}} < T_{\text{Computation}}$$

For **Stage 1**, the number of idle steps (bubbles) is **always** exactly the same, regardless of how many microbatches (\$M\$) you have.

$$2 \times (P - 1)$$

A rule of thumb:

$$M = 4P$$

Phase	→ Fwd Lane (1 → 2)	← Bwd Lane (2 → 1)	Total Load	Potential Bottleneck?
-------	-----------------------	-----------------------	------------	-----------------------

Warm-Up	● Heavy Traffic	● Empty	Half	No (Lane is clear)
Steady State	● Heavy Traffic	● Heavy Traffic	FULL	YES (Max Stress)
Cool-Down	● Empty	● Heavy Traffic	Half	No (Lane is clear)

The Forward Lane (\$1 \to 2\$): For sending Activations.

The Backward Lane (\$2 \to 1\$): For sending Gradients.

During Warm-Up and Cool-Down, half of the "Highway" (the bandwidth) is empty. Smart systems definitely try to fill that empty space. This is somehow wrong ●

The Good News:

Because they are separate roads, you don't *need* to wait for Warm-Up or Cool-Down. You should be prefetching data **100% of the time**, even during the busiest part of the Steady State. The traffic on Highway A (Pipeline) does not block Highway B (Data Loading). ●

Parameter Prefetching (ZeRO-3 / FSDP)

- **The Task:** In "Sharded" training (ZeRO-3), GPU 1 needs to fetch weights from GPU 2 to do math.
- **The Trick:**
 - **During Warm-Up:** We are doing Forward passes. The "Backward" lane (GPU \$2 \to 1\$) is empty.
 - **Action:** We can use that empty lane to **prefetch the weights needed for the Backward pass**.
 - **Result:** When the Backward pass finally starts, the weights are *already there*.